

AuditPlatform

Rapport d'analyse — plateforme-audit-ia-2

Généré le 23/04/2026 à 10:40

■ Informations du projet

Nom du projet	plateforme-audit-ia-2
URL du projet	yosrchiha01/plateforme-audit-ia-2
Branche analysée	main
Date de l'analyse	23/04/2026 10:07
Modèle LLM	llama-3.3-70b-versatile
Analyse OWASP	Activée

■ Scores

Qualité	75	75/100
Sécurité	15	15/100
Performance	35	35/100

■■ Vulnérabilités (71)

A07 — IDENTIFICATION AND AUTHENTICATION FAILURES
Sévérité : CRITIQUE
Fichier : frontend/app/forgot-password/page.tsx — ligne 24
Correction : Utiliser une méthode de vérification de mot de passe sécurisée, comme bcrypt, pour stocker les mots de passe
A07 — IDENTIFICATION AND AUTHENTICATION FAILURES
Sévérité : CRITIQUE

Fichier : frontend/app/login/page.tsx — ligne 21

Correction : Utiliser une méthode de vérification de mot de passe sécurisée, comme bcrypt, pour stocker les mots de passe

A07 — IDENTIFICATION AND AUTHENTICATION FAILURES

Sévérité : CRITIQUE

Fichier : frontend/app/profile/page.tsx — ligne 20

Correction : Utiliser une méthode de vérification de mot de passe sécurisée, comme bcrypt, pour stocker les mots de passe

A07 — IDENTIFICATION AND AUTHENTICATION FAILURES

Sévérité : CRITIQUE

Fichier : frontend/app/register/page.tsx — ligne 34

Correction : Utiliser une méthode de vérification de mot de passe sécurisée, comme bcrypt, pour stocker les mots de passe

A07 — IDENTIFICATION AND AUTHENTICATION FAILURES

Sévérité : CRITIQUE

Fichier : frontend/app/reset-password/page.tsx — ligne 37

Correction : Utiliser une méthode de vérification de mot de passe sécurisée, comme bcrypt, pour stocker les mots de passe

Maintenabilité

Sévérité : HAUTE

Fichier : frontend/app/Admin_page/page.tsx — ligne 0

Correction : Réorganiser le code pour séparer les responsabilités et améliorer la lisibilité.

Maintenabilité

Sévérité : HAUTE

Fichier : frontend/app/analyse/page.tsx — ligne 0

Correction : Réorganiser le code pour séparer les responsabilités et améliorer la lisibilité.

SOLID-S

Sévérité : HAUTE

Fichier : frontend/app/Admin_page/page.tsx — ligne 47

Correction : La fonction loadAll() est trop longue et complexe. Il est recommandé de la décomposer en plusieurs fonctions plus petites et plus faciles à tester.

SOLID-D

Sévérité : HAUTE

Fichier : frontend/app/Exploreformpage/page.tsx — ligne 16

Correction : La fonction handleSubmit() dépend directement de l'implémentation concrète de la fonction axios.post(). Il est recommandé de définir une interface pour la fonction axios.post() et de la passer en paramètre à la fonction handleSubmit().

SOLID-D

Sévérité : HAUTE

Fichier : frontend/app/analyse/page.tsx — ligne 65

Correction : La fonction lancerAnalyse() dépend directement de l'implémentation concrète de la fonction axios.post(). Il est recommandé de définir une interface pour la fonction axios.post() et de la passer en paramètre à la fonction lancerAnalyse().

Requêtes N+1

Sévérité : HAUTE

Fichier : frontend/app/codedepots/page.tsx — ligne 37

Correction : Utiliser la méthode `useQuery` de `react-query` pour gérer les requêtes API et éviter les requêtes N+1.

Requêtes N+1

Sévérité : HAUTE

Fichier : frontend/app/dashboard/page.tsx — ligne 57

Correction : Utiliser la méthode `useQuery` de `react-query` pour gérer les requêtes API et éviter les requêtes N+1.

Requêtes N+1

Sévérité : HAUTE

Fichier : frontend/app/depots/page.tsx — ligne 28

Correction : Utiliser la méthode `useQuery` de `react-query` pour gérer les requêtes API et éviter les requêtes N+1.

A03 — INJECTION

Sévérité : HAUTE

Fichier : frontend/app/difference/page.tsx — ligne 11

Correction : Utiliser une fonction de parsing de JSON sécurisée, comme `JSON.parse()` avec un try/catch pour gérer les erreurs de parsing

A02 — CRYPTOGRAPHIC FAILURES

Sévérité : HAUTE

Fichier : frontend/app/explorer/page.tsx — ligne 1

Correction : Utiliser une bibliothèque de cryptographie sécurisée, comme `crypto-js`, pour stocker et traiter les données sensibles

A02 — CRYPTOGRAPHIC FAILURES

Sévérité : HAUTE

Fichier : frontend/app/lib/api.ts — ligne 1

Correction : Utiliser une bibliothèque de cryptographie sécurisée, comme `crypto-js`, pour stocker et traiter les données sensibles

A08 — DATA INTEGRITY FAILURES

Sévérité : HAUTE

Fichier : frontend/app/profile/page.tsx — ligne 1

Correction : Utiliser une bibliothèque de désérialisation sécurisée, comme `json5`, pour désérialiser les données entrantes

A08 — DATA INTEGRITY FAILURES

Sévérité : HAUTE

Fichier : frontend/app/register/page.tsx — ligne 34

Correction : Utiliser une bibliothèque de désérialisation sécurisée, comme `json5`, pour désérialiser les données entrantes

A08 — DATA INTEGRITY FAILURES

Sévérité : HAUTE

Fichier : frontend/app/reset-password/page.tsx — ligne 37

Correction : Utiliser une bibliothèque de désérialisation sécurisée, comme json5, pour désérialiser les données entrantes

Gestion erreurs

Sévérité : HAUTE

Fichier : frontend/app/forgot-password/page.tsx — ligne 40

Correction : La fonction `handleSubmit` ne traite pas les exceptions génériques. Il est préférable d'utiliser des exceptions spécifiques pour améliorer la lisibilité et la maintenabilité du code.

Gestion erreurs

Sévérité : HAUTE

Fichier : frontend/app/login/page.tsx — ligne 20

Correction : La fonction `handleLogin` ne traite pas les exceptions génériques. Il est préférable d'utiliser des exceptions spécifiques pour améliorer la lisibilité et la maintenabilité du code.

Gestion erreurs

Sévérité : HAUTE

Fichier : frontend/app/register/page.tsx — ligne 33

Correction : La fonction `handleSubmit` ne traite pas les exceptions génériques. Il est préférable d'utiliser des exceptions spécifiques pour améliorer la lisibilité et la maintenabilité du code.

Gestion erreurs

Sévérité : HAUTE

Fichier : frontend/app/reset-password/page.tsx — ligne 51

Correction : La fonction `handleSubmit` ne traite pas les exceptions génériques. Il est préférable d'utiliser des exceptions spécifiques pour améliorer la lisibilité et la maintenabilité du code.

Docstring manquant

Sévérité : MOYENNE

Fichier : backend/app/config/database.py — ligne 0

Correction : Ajouter une docstring pour expliquer le rôle de la fonction `get_db()`

Commentaire absent

Sévérité : MOYENNE

Fichier : backend/app/routes/analyses.py — ligne 67

Correction : Ajouter un commentaire pour expliquer la logique de la fonction lancer_analyse()

Docstring manquant

Sévérité : MOYENNE

Fichier : backend/app/routes/depots.py — ligne 0

Correction : Ajouter une docstring pour expliquer le rôle de la fonction get_db()

Docstring manquant

Sévérité : MOYENNE

Fichier : backend/app/services/gitlab_client.py — ligne 0

Correction : Ajouter une docstring pour expliquer le rôle de la fonction get_gitlab_project()

Docstring manquant

Sévérité : MOYENNE

Fichier : backend/app/services/llm_service.py — ligne 0

Correction : Ajouter une docstring pour expliquer le rôle de la fonction analyser_code()

Complexité

Sévérité : MOYENNE

Fichier : frontend/app/Admin_page/page.tsx — ligne 73

Correction : Réduire la complexité de la fonction en utilisant des fonctions plus petites et plus faciles à comprendre.

Lisibilité

Sévérité : MOYENNE

Fichier : frontend/app/Exploreformpage/page.tsx — ligne 0

Correction : Utiliser des variables plus significatives et des noms de fonctions plus explicites.

Commentaire absent

Sévérité : MOYENNE

Fichier : frontend/app/Admin_page/page.tsx — ligne 73

Correction : Ajouter un commentaire pour expliquer l'algorithme utilisé dans la fonction DonutChart

Commentaire absent

Sévérité : MOYENNE

Fichier : frontend/app/Exploreformpage/page.tsx — ligne 22

Correction : Ajouter un commentaire pour expliquer l'algorithme utilisé dans la fonction handleSubmit

Commentaire absent

Sévérité : MOYENNE

Fichier : frontend/app/analyse/page.tsx — ligne 61

Correction : Ajouter un commentaire pour expliquer l'algorithme utilisé dans la fonction lancerAnalyse

Gestion erreurs

Sévérité : MOYENNE

Fichier : frontend/app/Admin_page/page.tsx — ligne 153

Correction : Il est recommandé de gérer les erreurs de manière plus spécifique, plutôt que de simplement ignorer les exceptions.

Gestion erreurs

Sévérité : MOYENNE

Fichier : frontend/app/analyse/page.tsx — ligne 91

Correction : Il est recommandé de gérer les erreurs de manière plus spécifique, plutôt que de simplement ignorer les exceptions.

Complexité

Sévérité : MOYENNE

Fichier : frontend/app/codedepots/page.tsx — ligne 38

Correction : Réduire la complexité de la fonction fetchDepots en utilisant des fonctions plus petites et plus faciles à comprendre.

Complexité

Sévérité : MOYENNE

Fichier : frontend/app/dashboard/page.tsx — ligne 35

Correction : Réduire la complexité de la fonction useEffect en utilisant des fonctions plus petites et plus faciles à comprendre.

Complexité

Sévérité : MOYENNE

Fichier : frontend/app/depots/page.tsx — ligne 23

Correction : Réduire la complexité de la fonction fetchDepots en utilisant des fonctions plus petites et plus faciles à comprendre.

SELECT *

Sévérité : MOYENNE

Fichier : frontend/app/codedepots/page.tsx — ligne 45

Correction : Sélectionner uniquement les colonnes nécessaires au lieu de sélectionner toutes les colonnes avec `SELECT \*`.

SELECT *

Sévérité : MOYENNE

Fichier : frontend/app/dashboard/page.tsx — ligne 65

Correction : Sélectionner uniquement les colonnes nécessaires au lieu de sélectionner toutes les colonnes avec `SELECT \*`.

SELECT *

Sévérité : MOYENNE

Fichier : frontend/app/depots/page.tsx — ligne 36

Correction : Sélectionner uniquement les colonnes nécessaires au lieu de sélectionner toutes les colonnes avec `SELECT \*`.

A05 — SECURITY MISCONFIGURATION

Sévérité : MOYENNE

Fichier : frontend/app/login/page.tsx — ligne 21

Correction : Utiliser un en-tête HTTP de sécurité, comme Content-Security-Policy, pour protéger les ressources externes

Boucle $O(n^2)$

Sévérité : MOYENNE

Fichier : frontend/app/difference/page.tsx — ligne 306

Correction : Utiliser une fonction de recherche linéaire ou un index/dictionnaire pour améliorer la complexité de l'algorithme.

Boucle $O(n^2)$

Sévérité : MOYENNE

Fichier : frontend/app/explorer/page.tsx — ligne 305

Correction : Utiliser une fonction de recherche linéaire ou un index/dictionnaire pour améliorer la complexité de l'algorithme.

Boucle $O(n^2)$

Sévérité : MOYENNE

Fichier : frontend/app/profile/page.tsx — ligne 327

Correction : Utiliser une fonction de recherche linéaire ou un index/dictionnaire pour améliorer la complexité de l'algorithme.

Ressource

Sévérité : MOYENNE

Fichier : frontend/app/layout.tsx — ligne 0

Correction : Utiliser un gestionnaire de contexte pour fermer les ressources ouvertes après utilisation.

Ressource

Sévérité : MOYENNE

Fichier : frontend/app/lib/api.ts — ligne 0

Correction : Utiliser un gestionnaire de contexte pour fermer les ressources ouvertes après utilisation.

Ressource

Sévérité : MOYENNE

Fichier : frontend/app/login/page.tsx — ligne 0

Correction : Utiliser un gestionnaire de contexte pour fermer les ressources ouvertes après utilisation.

Ressource

Sévérité : MOYENNE

Fichier : frontend/app/page.tsx — ligne 0

Correction : Utiliser un gestionnaire de contexte pour fermer les ressources ouvertes après utilisation.

Ressource

Sévérité : MOYENNE

Fichier : frontend/app/profile/page.tsx — ligne 0

Correction : Utiliser un gestionnaire de contexte pour fermer les ressources ouvertes après utilisation.

Ressource

Sévérité : MOYENNE

Fichier : frontend/app/register/page.tsx — ligne 0

Correction : Utiliser un gestionnaire de contexte pour fermer les ressources ouvertes après utilisation.

Ressource

Sévérité : MOYENNE

Fichier : frontend/app/reset-password/page.tsx — ligne 0

Correction : Utiliser un gestionnaire de contexte pour fermer les ressources ouvertes après utilisation.

Docstring manquant

Sévérité : MOYENNE

Fichier : frontend/app/difference/page.tsx — ligne 4

Correction : Ajouter une docstring pour expliquer le rôle de la fonction DifferencePage

Commentaire absent

Sévérité : MOYENNE

Fichier : frontend/app/explorer/page.tsx — ligne 27

Correction : Ajouter un commentaire pour expliquer la fonction buildTree

Commentaire absent

Sévérité : MOYENNE

Fichier : frontend/app/login/page.tsx — ligne 15

Correction : Ajouter un commentaire pour expliquer la fonction handleLogin

Commentaire absent

Sévérité : MOYENNE

Fichier : frontend/app/profile/page.tsx — ligne 17

Correction : Ajouter un commentaire pour expliquer la fonction useEffect

Commentaire absent

Sévérité : MOYENNE

Fichier : frontend/app/register/page.tsx — ligne 19

Correction : Ajouter un commentaire pour expliquer la fonction handleSubmit

Commentaire absent

Sévérité : MOYENNE

Fichier : frontend/app/reset-password/page.tsx — ligne 19

Correction : Ajouter un commentaire pour expliquer la fonction handleSubmit

SOLID-S

Sévérité : MOYENNE

Fichier : frontend/app/difference/page.tsx — ligne 11

Correction : La fonction `try`-`catch`` est utilisée pour gérer les erreurs, mais elle ne traite pas les exceptions génériques. Il est préférable d'utiliser des exceptions spécifiques pour améliorer la lisibilité et la maintenabilité du code.

SOLID-I

Sévérité : MOYENNE

Fichier : frontend/app/explorer/page.tsx — ligne 34

Correction : L'interface `FileItem` contient plusieurs propriétés qui ne sont pas nécessaires pour toutes les implémentations. Il est préférable de séparer ces propriétés en plusieurs interfaces pour améliorer la flexibilité et la maintenabilité du code.

SOLID-D

Sévérité : MOYENNE

Fichier : frontend/app/profile/page.tsx — ligne 65

Correction : La fonction `handleEditSave` dépend de la fonction `axios.put` qui est une implémentation concrète. Il est préférable d'utiliser une abstraction pour améliorer la flexibilité et la maintenabilité du code.

Docstring manquant

Sévérité : FAIBLE

Fichier : frontend/app/Admin_page/page.tsx — ligne 44

Correction : Ajouter une docstring pour expliquer le rôle de la fonction `DonutChart`

Nommage ambigu

Sévérité : FAIBLE

Fichier : frontend/app/Admin_page/page.tsx — ligne 120

Correction : Renommer la variable `color` en `backgroundColor` pour clarifier son rôle

Docstring manquant

Sévérité : FAIBLE

Fichier : frontend/app/Exploreformpage/page.tsx — ligne 16

Correction : Ajouter une docstring pour expliquer le rôle de la fonction `handleSubmit`

Docstring manquant

Sévérité : FAIBLE

Fichier : frontend/app/analyse/page.tsx — ligne 45

Correction : Ajouter une docstring pour expliquer le rôle de la fonction `lancerAnalyse`

Lisibilité

Sévérité : FAIBLE

Fichier : frontend/app/codedepots/page.tsx — ligne 0

Correction : Considérer l'utilisation de fonctions plus petites et plus faciles à comprendre pour améliorer la lisibilité du code.

Lisibilité

Sévérité : FAIBLE

Fichier : frontend/app/dashboard/page.tsx — ligne 0

Correction : Considérer l'utilisation de fonctions plus petites et plus faciles à comprendre pour améliorer la lisibilité du code.

Lisibilité

Sévérité : FAIBLE

Fichier : frontend/app/depots/page.tsx — ligne 0

Correction : Considérer l'utilisation de fonctions plus petites et plus faciles à comprendre pour améliorer la lisibilité du code.

SELECT *

Sévérité : FAIBLE

Fichier : frontend/app/register/page.tsx — ligne 34

Correction : Sélectionner uniquement les colonnes nécessaires pour améliorer les performances de la requête SQL.

SELECT *

Sévérité : FAIBLE

Fichier : frontend/app/reset-password/page.tsx — ligne 37

Correction : Sélectionner uniquement les colonnes nécessaires pour améliorer les performances de la requête SQL.

Nommage ambigu

Sévérité : FAIBLE

Fichier : frontend/app/forgot-password/page.tsx — ligne 0

Correction : Le nom de la fonction ForgotPassword n'est pas très explicite, pensez à le renommer en quelque chose comme resetPassword

■ Recommandations

1. Ajouter des commentaires pour expliquer la logique des fonctions

Les commentaires sont importants pour expliquer la logique des fonctions et faciliter la maintenance du code.

2. Utiliser des noms de variables et de fonctions clairs

Les noms de variables et de fonctions doivent être clairs et précis pour faciliter la compréhension du code.

3. Utiliser des types de données pour les variables

Les types de données doivent être utilisés pour les variables pour faciliter la compréhension du code et éviter les erreurs.

4. Réduire la complexité de la fonction

Réduire la complexité de la fonction en utilisant des fonctions plus petites et plus faciles à comprendre. Cela améliorera la lisibilité et la maintenabilité du code.

5. Réorganiser le code

Réorganiser le code pour séparer les responsabilités et améliorer la lisibilité. Cela améliorera la maintenabilité et la compréhension du code.

6. Utiliser des variables plus significatives

Utiliser des variables plus significatives et des noms de fonctions plus explicites. Cela améliorera la lisibilité et la compréhension du code.

7. Ajouter des commentaires explicatifs

Ajouter des commentaires pour expliquer les algorithmes utilisés dans les fonctions et les variables pour améliorer la lisibilité du code

8. Améliorer la documentation

Ajouter des docstrings pour expliquer le rôle des fonctions et des variables pour améliorer la lisibilité du code

9. Renommer les variables

Renommer les variables pour clarifier leur rôle et améliorer la lisibilité du code

10. Décomposer les fonctions longues et complexes

Les fonctions longues et complexes sont difficiles à tester et à maintenir. Il est recommandé de les décomposer en plusieurs fonctions plus petites et plus faciles à tester.

11. Gérer les erreurs de manière plus spécifique

Les erreurs doivent être gérées de manière plus spécifique, plutôt que de simplement ignorer les exceptions. Cela permet de mieux comprendre les problèmes et de les résoudre plus efficacement.

12. Définir des interfaces pour les fonctions dépendantes

Les fonctions dépendantes doivent être définies avec des interfaces pour éviter les dépendances directes à des implémentations concrètes. Cela permet de mieux comprendre les dépendances et de les gérer de manière plus efficace.

13. Réduire la complexité des fonctions

Les fonctions trop complexes peuvent être difficiles à comprendre et à maintenir. Considérez la réduction de la complexité en utilisant des fonctions plus petites et plus faciles à comprendre.

14. Améliorer la lisibilité du code

La lisibilité du code est importante pour les développeurs qui doivent comprendre et maintenir le code. Considérez l'utilisation de fonctions plus petites et plus faciles à comprendre pour améliorer la lisibilité du

code.

15. Utiliser la méthode `useQuery` de `react-query` pour gérer les requêtes API

La méthode `useQuery` de `react-query` permet de gérer les requêtes API de manière efficace et évite les requêtes N+1. Elle permet également de gérer les erreurs et les réessais de requêtes.

16. Sélectionner uniquement les colonnes nécessaires

Sélectionner uniquement les colonnes nécessaires au lieu de sélectionner toutes les colonnes avec `SELECT *` peut améliorer les performances de la requête et réduire la charge sur la base de données.

17. Utiliser des méthodes de vérification de mot de passe sécurisées

Utiliser des méthodes de vérification de mot de passe sécurisées, comme `bcrypt`, pour stocker les mots de passe et éviter les attaques par force brute.

18. Utiliser des bibliothèques de cryptographie sécurisées

Utiliser des bibliothèques de cryptographie sécurisées, comme `crypto-js`, pour stocker et traiter les données sensibles et éviter les attaques par déni de service.

19. Utiliser des en-têtes HTTP de sécurité

Utiliser des en-têtes HTTP de sécurité, comme `Content-Security-Policy`, pour protéger les ressources externes et éviter les attaques par injection de code.

20. Utiliser des bibliothèques de désérialisation sécurisées

Utiliser des bibliothèques de désérialisation sécurisées, comme `json5`, pour désérialiser les données entrantes et éviter les attaques par injection de code.

21. Utiliser des gestionnaires de contexte pour fermer les ressources ouvertes

Utilisez des gestionnaires de contexte pour fermer les ressources ouvertes après utilisation, ce qui peut améliorer les performances et réduire les risques de bugs.

22. Utiliser des fonctions de recherche linéaire ou des index/dictionnaires

Utilisez des fonctions de recherche linéaire ou des index/dictionnaires pour améliorer la complexité de l'algorithme et réduire les risques de bugs.

23. Ajouter des docstrings pour les fonctions

Ajoutez des docstrings pour expliquer le rôle de chaque fonction, cela aidera les autres développeurs à comprendre le code.

24. Ajouter des commentaires pour les blocs de logique

Ajoutez des commentaires pour expliquer les blocs de logique complexes, cela aidera les autres développeurs à comprendre le code.

25. Renommer les fonctions pour les rendre plus explicites

Renommez les fonctions pour les rendre plus explicites, cela aidera les autres développeurs à comprendre le code.

26. Utiliser des exceptions spécifiques

Les exceptions génériques (par exemple, `Error` ou `Exception`) ne fournissent pas d'informations précises sur les erreurs qui se produisent. Il est préférable d'utiliser des exceptions spécifiques pour améliorer la lisibilité et la maintenabilité du code.

27. Séparer les propriétés dans les interfaces

Les interfaces peuvent contenir plusieurs propriétés qui ne sont pas nécessaires pour toutes les implémentations. Il est préférable de séparer ces propriétés en plusieurs interfaces pour améliorer la flexibilité et la maintenabilité du code.

28. Utiliser des abstractions pour les dépendances

Les fonctions qui dépendent de la fonction ``axios.put`` sont trop spécifiques et ne sont pas faciles à tester. Il est préférable d'utiliser une abstraction pour améliorer la flexibilité et la maintenabilité du code.